

String, Date, and Numeric Function Quick Reference

String, Date, and Numeric Function Quick Reference

Chapter 10 — Cleaning and Transforming Real-World Data

Keep this document open alongside your SQL editor. Every function covered in Chapter 10 is listed alphabetically with its syntax, key parameters, and a one-line working example you can run immediately.

How to Use This Reference

- Syntax lines use UPPER_CASE for SQL keywords and lower_case for your own values.
- All examples assume a table called customers or orders consistent with Chapter 10 exercises.
- Functions behave identically inside SELECT, WHERE, and ORDER BY clauses unless noted.
- Dialect notes flag where T-SQL (SQL Server) and MySQL differ.

Alphabetical Function Reference

Function	Category	One-Line Purpose
ABS	Numeric	Returns the absolute (non-negative) value of a number
CAST	Conversion	Converts a value from one data type to another
CEILING	Numeric	Rounds a number up to the nearest integer
CONCAT	String	Joins two or more strings into one
DATEADD	Date	Adds an interval to a date
DATEDIFF	Date	Returns the difference between two dates
DATEPART	Date	Extracts a single component (year, month, day, etc.) from a date
DAY	Date	Returns the day portion of a date
FLOOR	Numeric	Rounds a number down to the nearest integer
LOWER	String	Converts all characters to lowercase
MONTH	Date	Returns the month portion of a date
REPLACE	String	Substitutes every occurrence of a substring with another string

ROUND	Numeric	Rounds a number to a specified number of decimal places
SUBSTRING	String	Extracts a portion of a string by position and length
TRIM	String	Removes leading and trailing whitespace
UPPER	String	Converts all characters to uppercase
YEAR	Date	Returns the year portion of a date

Detailed Entries

ABS

Syntax: ABS(numeric_expression)

Parameters: numeric_expression — any column or value that returns a number.

Example: SELECT ABS(balance) FROM accounts; — returns 250 instead of -250.

CAST

Syntax: CAST(expression AS data_type)

Parameters: expression — the value to convert; data_type — target type such as INT, VARCHAR(50), DATE, DECIMAL(10,2).

Example: SELECT CAST(order_total AS DECIMAL(10,2)) FROM orders; — forces a string price column into a numeric type for arithmetic.

CEILING

Syntax: CEILING(numeric_expression)

Example: SELECT CEILING(4.1) FROM orders; — returns 5.

CONCAT

Syntax: CONCAT(string1, string2 [, stringN ...])

Parameters: Accepts two or more string values or columns; NULL values are silently ignored in most dialects.

Example: SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM customers;

DATEADD

Syntax: DATEADD(interval, number, date) (T-SQL / SQL Server)

Parameters: interval — unit such as DAY, MONTH, YEAR; number — integer offset (negative moves backward); date — starting date column or literal.

Example: `SELECT DATEADD(DAY, 30, order_date) AS due_date FROM orders;` — calculates a 30-day payment deadline.

DATEDIFF

Syntax: `DATEDIFF(interval, start_date, end_date)`

Example: `SELECT DATEDIFF(DAY, signup_date, GETDATE()) AS days_active FROM customers;` — counts days since each customer signed up.

DATEPART

Syntax: `DATEPART(interval, date)`

Example: `SELECT DATEPART(MONTH, order_date) AS order_month FROM orders;` — returns 3 for March.

DAY / MONTH / YEAR

Syntax: `DAY(date) | MONTH(date) | YEAR(date)`

Example: `SELECT YEAR(order_date) AS yr, MONTH(order_date) AS mo FROM orders;` — extracts components for time-series grouping.

FLOOR

Syntax: `FLOOR(numeric_expression)`

Example: `SELECT FLOOR(9.9) FROM orders;` — returns 9.

LOWER / UPPER

Syntax: `LOWER(string_expression) | UPPER(string_expression)`

Example: `SELECT UPPER(country) FROM customers;` — standardises 'usa', 'USA', and 'Usa' to a single format for reliable GROUP BY.

REPLACE

Syntax: `REPLACE(string_expression, search_string, replacement_string)`

Example: `SELECT REPLACE(phone, '-', '') FROM customers;` — strips hyphens from phone numbers.

ROUND

Syntax: `ROUND(numeric_expression, decimal_places)`

Parameters: decimal_places — use 0 for whole numbers; negative values round to tens, hundreds, etc.

Example: `SELECT ROUND(unit_price, 2) FROM orders;` — trims a calculated price to two decimal places.

SUBSTRING

Syntax: `SUBSTRING(string_expression, start_position, length)`

Parameters: start_position — 1-based index of the first character to extract; length — number of characters to return.

Example: `SELECT SUBSTRING(product_code, 1, 3) AS category_code FROM products;` — pulls the first three characters of a product code.

TRIM

Syntax: `TRIM(string_expression)`

Example: `SELECT TRIM(email) FROM customers;` — removes invisible leading/trailing spaces that cause failed JOIN matches.

Quick Reminder: CASE WHEN for Conditional Columns

Although not a single function, CASE WHEN pairs with every function above to create bucketed columns:

```
SELECT
    customer_id,
    ROUND(total_spent, 2) AS spend,
    CASE
        WHEN total_spent >= 1000 THEN 'High'
        WHEN total_spent >= 500  THEN 'Medium'
        ELSE 'Low'
    END AS spend_tier
FROM orders;
```

Use this pattern whenever a numeric or date function produces a value you need to categorise for reporting.